UNIT 22 90 min

# Capstone and Responsible Data Work

Plan and carry out a complete data project from question to report, and check it for leakage, bias, and privacy before anyone trusts it.

---

**By the end of this unit you will be able to**

- Turn a vague question into a measurable prediction task
- Run an end-to-end project: load, clean, explore, model, evaluate, report
- Check a project for leakage, misleading charts, and unfair outcomes
- Protect personal data, including student records
- Write a short model card that states what a model is and is not for

## The project

This unit is a project, not a lecture. You will use the skills from Units 1 through 21 on one question you care about. A useful project answers a question someone will act on, uses data you have the right to use, and reports both what the model does well and where it fails.

## Step 1: write the question

A question is measurable when you can say what counts as success. "Can we predict which students will need tutoring?" is vague. "Can we flag students whose quarter grade will fall below 70, using only data available by week 4?" is measurable: the target is defined, and the time limit prevents using information from the future.

- **Target:** what exactly are you predicting, and how is it measured?
- **Timing:** what information will be available at prediction time? Anything later is leakage.
- **Action:** what will someone do with the answer? If nothing, the project is an exercise.

## Step 2: check the data

project_start.py

```
import io
import pandas as pd

# Replace this text with pd.read_csv("your_file.csv") for real data.
raw = io.StringIO("""student,week1_quiz,week2_quiz,hw_done,final_grade
A01,88,91,5,89
A02,62,58,2,64
A03,75,,4,78
A04,93,95,5,96
A05,50,47,1,55
""")
df = pd.read_csv(raw)
print(df.info())
print(df.isna().sum())
print(df.describe().round(1))
```

Before modeling, look at missing values, impossible numbers (a quiz score of 140), duplicated rows, and the balance of the target. A model trained on a dataset that is 98% one class can score well by ignoring the rare class.

## Step 3: avoid leakage

**Leakage** means a feature contains information you would not have when making the prediction. The most common kind is a column that is recorded after the outcome, such as "final exam taken" when predicting the final grade. Leakage gives scores that look excellent and fail the first time the model meets real data. Remove leaking columns before you split, and keep the split before any fitting, as in Unit 19.

> **Watch out**
> A project that reports 99% accuracy on its first try should be checked for leakage before it is celebrated.

## Step 4: model and compare

Start with a simple baseline: always predict the most common class, or predict the average. Any real model must beat the baseline clearly. Then try one or two models, such as logistic regression and a decision tree, using the same split. Report the baseline next to every result.

**baseline.py**

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, random_state=0, stratify=data.targe
)

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
model = LogisticRegression(max_iter=5000).fit(X_train, y_train)

print("baseline:", round(baseline.score(X_test, y_test), 3))
print("logistic:", round(model.score(X_test, y_test), 3))
```

The breast cancer dataset ships with scikit-learn and is used here only to practice the workflow. It is not a clinical tool.

## Step 5: report honestly

A good report has five parts: the question, the data and its limits, the method, the results against the baseline, and the failures. Show at least one chart, labeled as Unit 18 taught. State uncertainty. Say what you did not test.

## Step 6: check fairness

A model can be accurate overall and still fail a group. Compute the same metric separately for each group that matters (for example, each class period, gender, or language background). If the results

differ a lot, find out why before deploying. Often the cause is a training set that under-represents the group.

by_group.py

```
import pandas as pd

results = pd.DataFrame({
    "group": ["A", "A", "A", "B", "B", "B"],
    "correct": [1, 1, 0, 1, 0, 0]
})
print(results.groupby("group")["correct"].mean().round(2))
```

## Privacy: student and personal data

Student records are sensitive, and in the United States they are protected by law (FERPA). The same caution applies to any personal data. Practical habits:

- Use the minimum data you need. Drop names and IDs before analysis, and keep a separate key only if you must link results back.
- Do not upload identifiable data to a public tool, notebook, or chatbot. Check your school's approved tools first.
- Share aggregated results (counts, averages with enough people in each group), not individual rows.
- Delete working copies you no longer need, and keep the original in its approved location.

> **Tip**
> Before using any dataset, ask: who collected it, for what purpose, and did those people agree to this use? If you cannot answer, do not use it yet.

## Step 7: write a model card

A **model card** is a short document that travels with a model. It answers: what the model predicts, what data it was trained on, how it was evaluated, where it works well, where it fails, and what it must not be used for. Writing it forces you to be specific. A template you can copy:

| Section | What to write |
|---|---|
| Purpose | The question, and the decision it supports |
| Data | Source, date range, size, known gaps |
| Method | Model type, features, split, random seed |
| Results | Metric on the held-out test set, compared with the baseline |
| Groups | Results for each important group, and any gaps |
| Limits | Where it fails; what it must not be used for |
| Owner | Who maintains it, and when it will be reviewed |

## The capstone checklist

1. A measurable question, with target, timing, and action written down
2. Data loaded, inspected, and cleaned; limits noted
3. Leaking columns removed; split done before any fitting
4. Baseline reported next to every model
5. Test set used once, at the end, for the final number
6. At least one labeled chart
7. Group results checked, with gaps explained
8. Personal data removed or protected
9. Model card written

## Practice

▼ **Exercise 1 — Find the leak**

A team predicts whether a loan will default. Their features include `days_late_on_loan` and `default_flag`. Which column leaks the answer, and why? Write the one-line fix.

▼ **Hint**

Ask whether you could know this value at the moment of the decision.

**Discussion**

1. Which is worse in your context: a false alarm (flagging a student who was fine) or a miss (not flagging one who needed help)? How does that choice change the model you should build?
2. Your model is 90% accurate overall, but 60% accurate for one group. What do you do before deploying?
3. Who should sign off on a model that makes decisions about people, and what should they see first?

## Key terms

**Leakage**

When a model is trained on information that would not be available at prediction time, inflating its score.

**Baseline**

A simple reference model, such as always predicting the most common class, that any real model must beat.

**Model card**

A short document describing a model's purpose, data, evaluation, limits, and owner.

**Fairness (group metrics)**

Checking that a model's performance is comparable across the groups it affects.

**Unsupervised learning**

Learning structure from unlabeled data, such as clustering similar examples together.

**FERPA**

The US federal law protecting the privacy of student education records.

**Aggregation**

Reporting summaries (counts, averages) instead of individual records, to protect privacy.

See all terms in the glossary →